

fox2db — controller model (Sofar / Waveshare ESP32-S3)

Load-following PV-surplus controller for a solar plant with two battery stores. It switches a quantized electrical load (EBox) so that the grid exchange stays near zero ("sweet spot"), without disturbing the autonomous Sofar battery.

Classification

Discrete-time, nonlinear hybrid automaton (Mealy machine with hysteresis) — NOT a PID and NOT an LTI system. The actuator is discrete (8 power levels via 3 relay bits), hence dead-band + rate limiting instead of a continuous control signal. Cycle: 60 s.

Inputs and outputs

```
Inputs :  pcc  grid exchange [W]    (+ = export)
          ebox  measured EBox load [W]
          bat1  Sofar battery power [W]  (autonomous, read-only)
          (SOC1 is NOT used; SOC2 drives separate guards)

State  :  s     relay state 0..7
          c     stabilization counter
          E_-1  previous cycle surplus (for trend)

Output :  s     one state 0..7 ->  EBox power P[s]
```

Plant limits

```
fuse 3x50A @230V ..... 34500 W grid connection
PV peak power ..... 35000 W
EBox max (state 7) ..... 11400 W (= 33% fuse, 33% PV)
Bat1 (Sofar) ..... 5 kWh
Bat2 (EBox) ..... 30 kWh (@state7: +0.63 %/min)
```

Mathematical model

Power map and rank

$$P = (0, 3000, 3650, 6650, 3900, 7100, 7800, 11400) \text{ W}$$

$$\Pi \text{ (ascending)} = (0, 3000, 3650, 3900, 6650, 7100, 7800, 11400)$$

Step recursion $x_k=(s,c,E)$

$$e_{\text{eff}} = \max(e_k, P_{s_{k-1}}) \quad (s_{k-1} > 0)$$

$$E_k = p_k + e_{\text{eff}} + b_k$$

$$\tau_k = Q(E_k + G) \quad (0 = \text{insufficient})$$

$$Q(B) = \text{highest state } s \text{ with } P_s \leq B$$

Up-ramp (max. one power step per cycle)

$$\tilde{\tau}_k = \min(\tau_k, \hat{s}(\text{rank}(s_{k-1}) + 1))$$

Switching / protection logic (order = priority)

EMERGENCY_FORCE	:	pcc < -1020 (=- (G+120)) & down	-> switch now
SWEET_SPOT_HOLD	:	up & pcc < 160	-> hold
TREND_BLOCK	:	up & dE/dt < -20 W/s	-> hold
BAT_GUARD_BLOCK	:	up & bat1 < -110 W	-> hold
STABILIZING	:	down & c < 2 cycles	-> hold
HYSTERESIS	:	down & dP < 505 W	-> hold

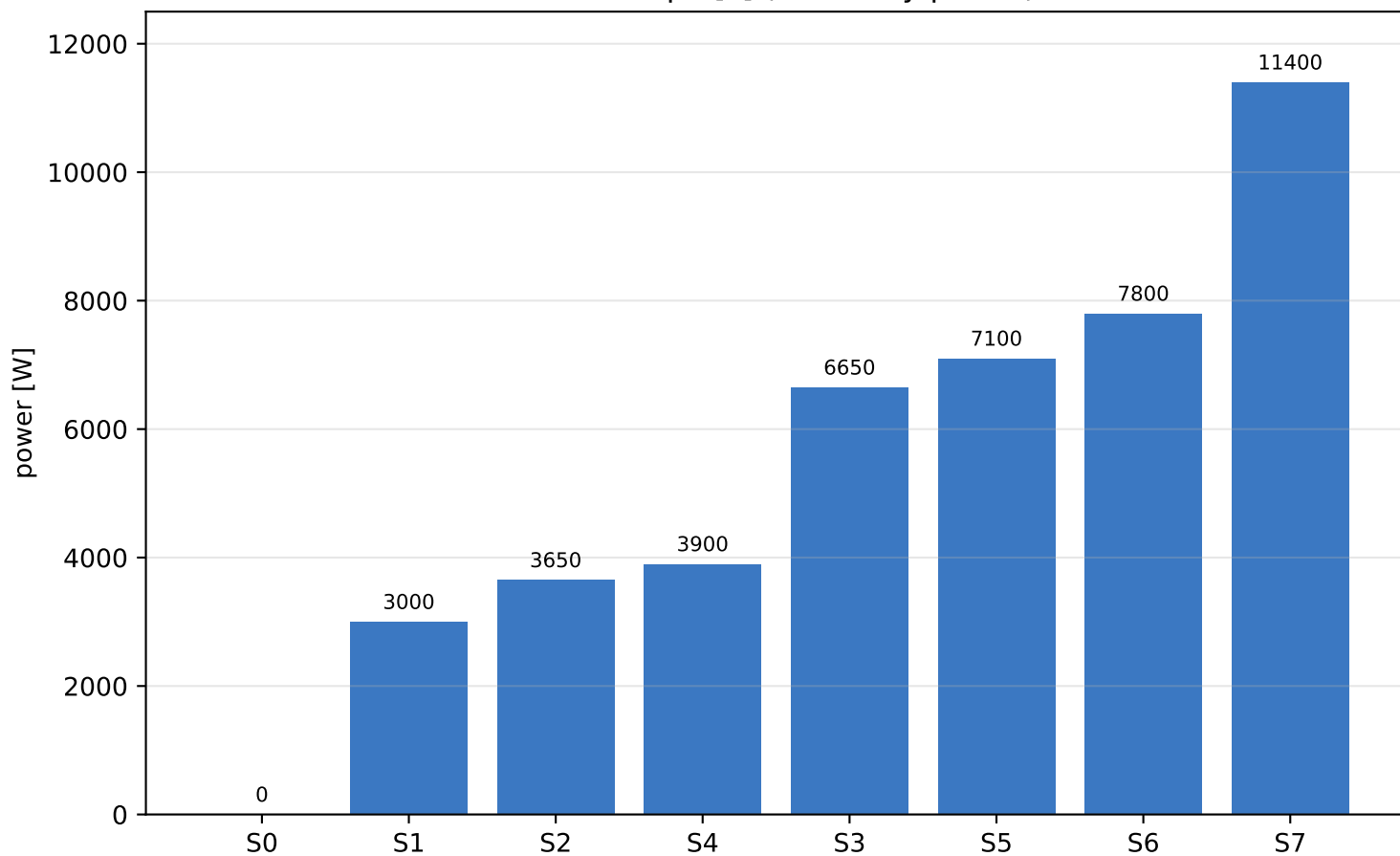
Constants

$$G = 900, H = 505, N = 2, \text{ EMERG} = G + 120 \text{ [W]}$$

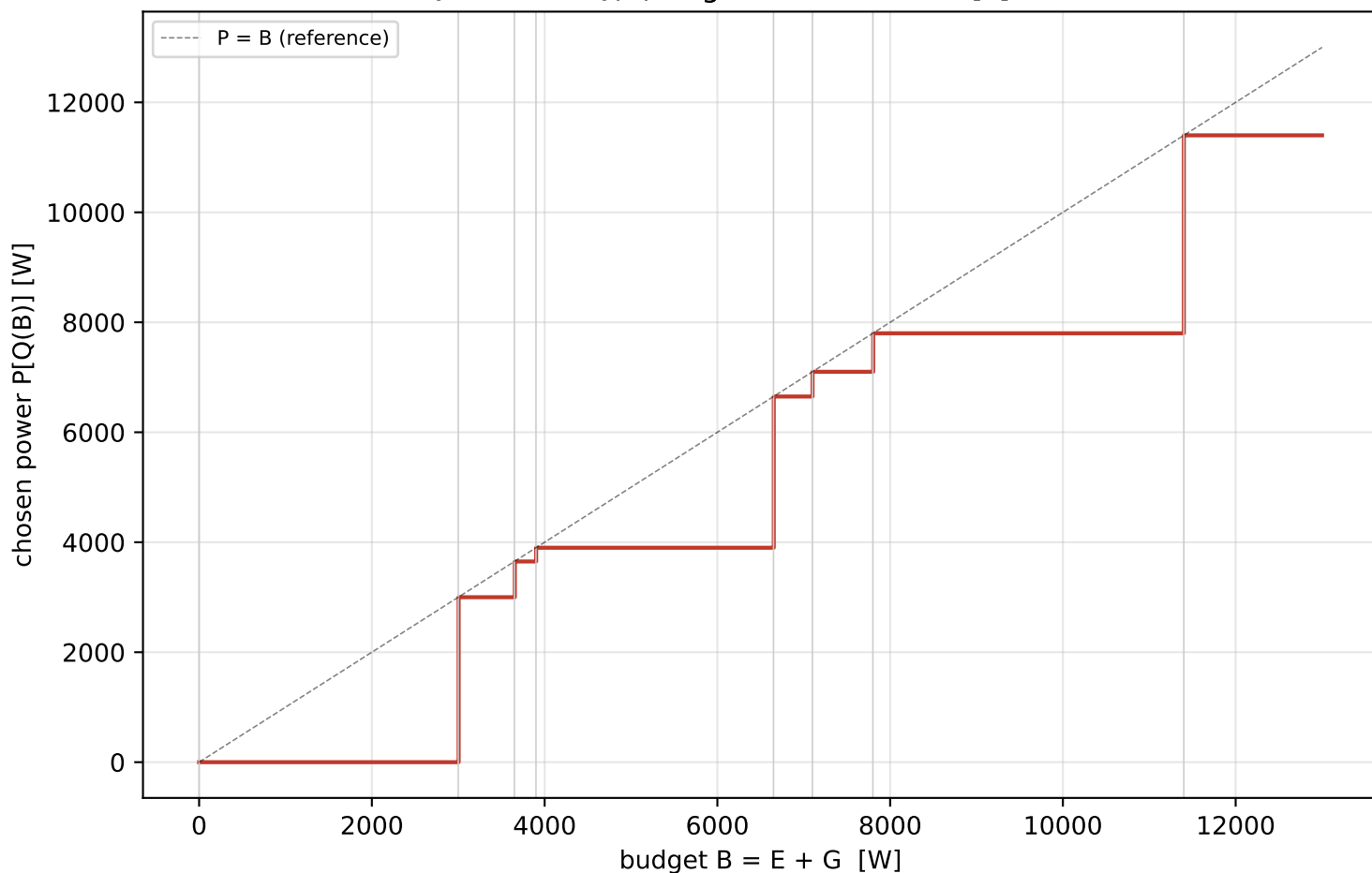
Dead-band (equilibrium, load on $e_{\text{eff}}=P_s$)

$$-G < p < (\Pi^+(s) - P_s) - G$$

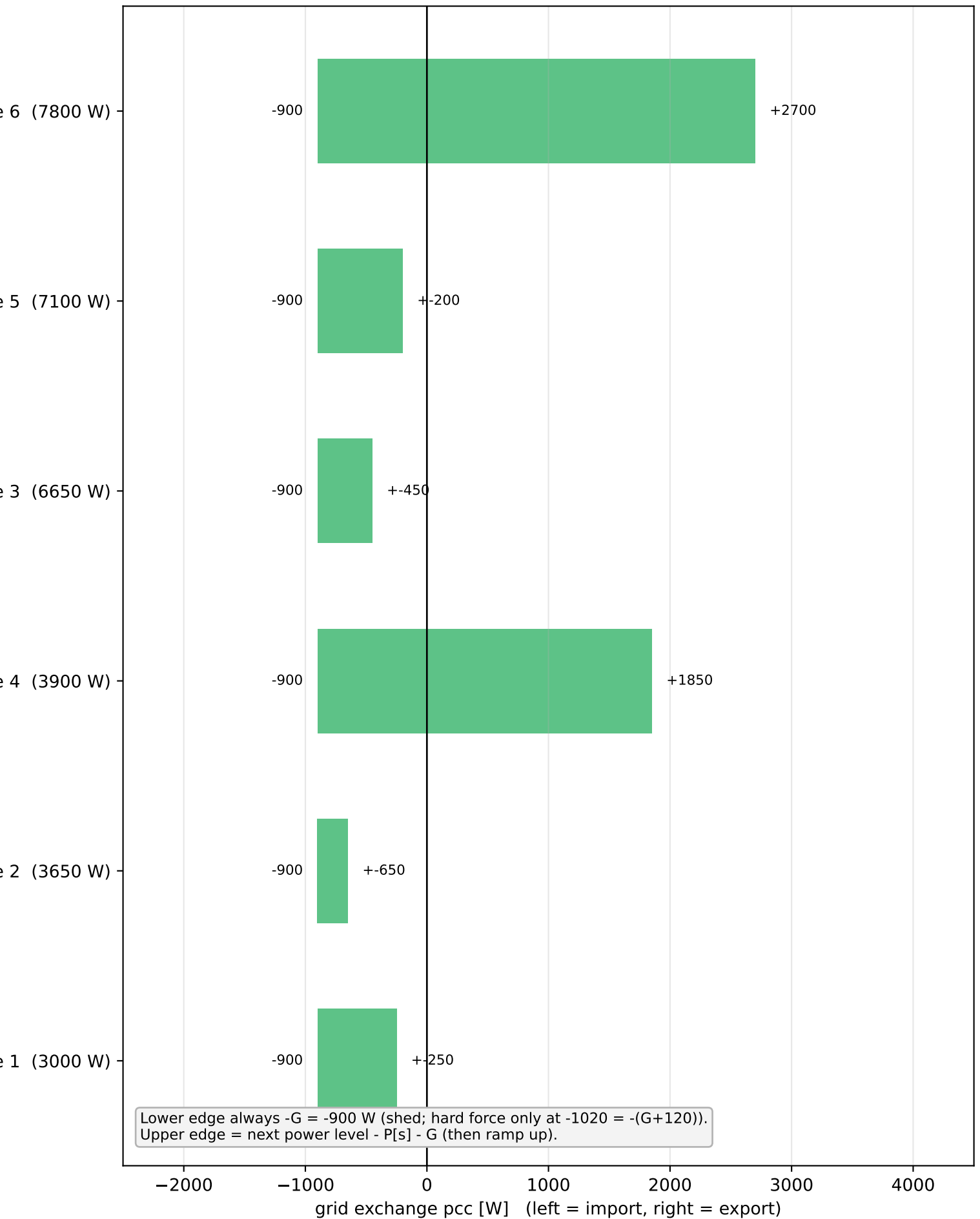
Power map $P[s]$ (sorted by power)



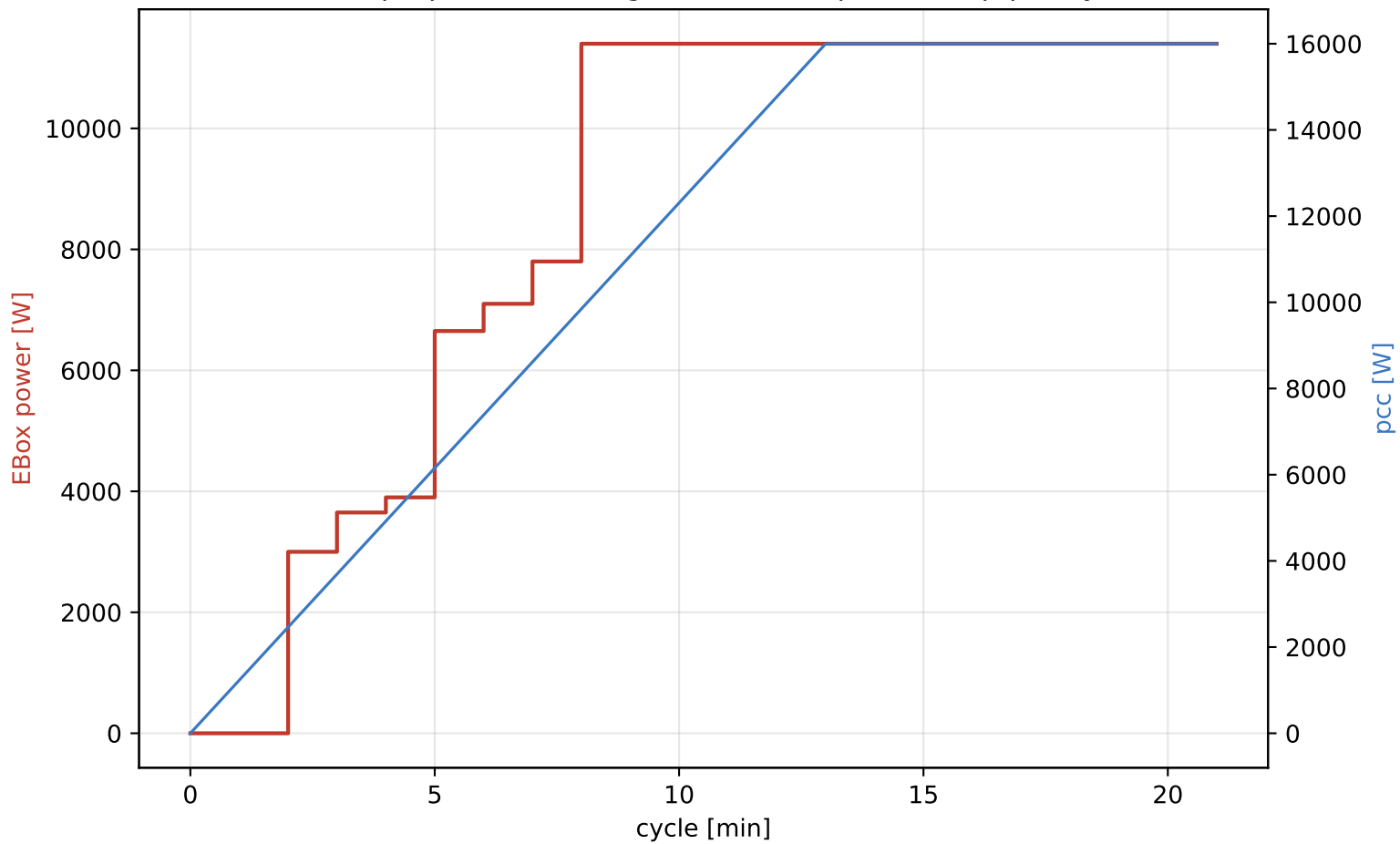
Quantizer $Q(B)$: highest state with $P[s] \leq B$



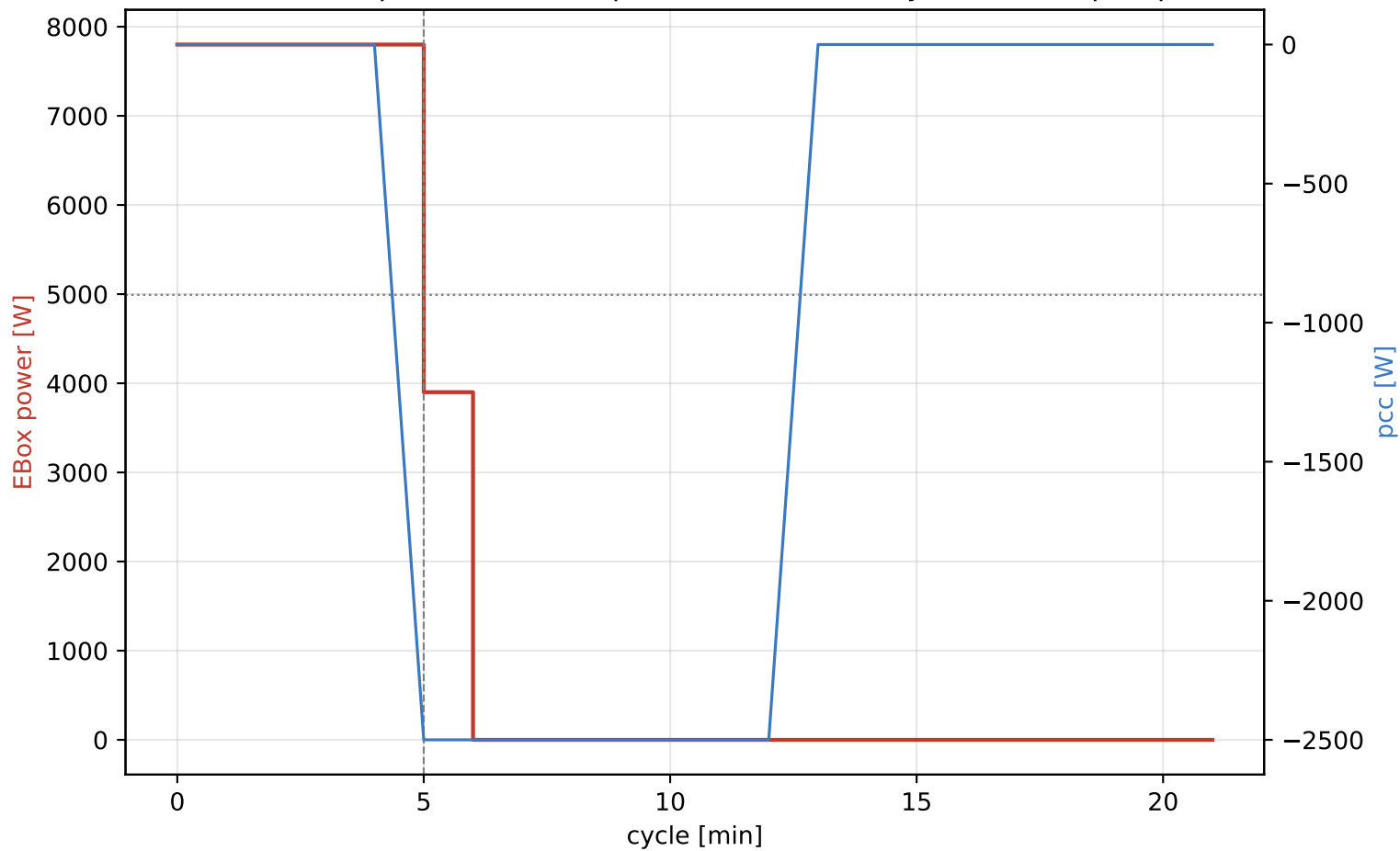
Dead-band (sweet spot): pcc range in which a state is held
(load on, ebox = P[s])



(a) Ramp-up: rate limiting — max. one power step per cycle



(b) Load step in the sweet spot: -2500 W from cycle 5 (heat pump)



Validation against real data

The reference `step_full_literal` was checked one-step-ahead against the running ESP's actual decisions (table `pv_decision_log`, version='waveshare'): each cycle (`state_from`, `pcc`, `bat1`, `ebox`) is fed in and the predicted next state compared.

Result (window 2026-06-10, midday; firmware before this tuning)

```
one-step state match   : 55 / 55   (100.0 %)
excess formula exact   : 54 / 55   ( 98.2 %)
reason + modifier      : 54 / 55   ( 98.2 %)
covered decisions: POWER_MATCHING, INSUFFICIENT_EXCESS,
RAMP_LIMITED, SWEET_SPOT_HOLD, TREND_BLOCK, BAT_GUARD_BLOCK,
EMERGENCY_FORCE, STABILIZING
```

Re-validate against post-tuning rows only (do not mix parameter sets).

Implementation note

The C code (`fox2db_logic.h`) compares by STATE NUMBER for the up-ramp rather than the power RANK. Since P is not monotone in the number (`state4=3900W < state3=6650W`), this deviates in edge cases from the rank-monotone model. `step_full_literal` mirrors the C code 1:1; `step_full_rank` is the clean form.

Artifacts

<code>fox2db_logic.h</code>	ESP control logic (C++)
<code>fox2dbEasy.py</code>	1:1 Python port (shadow)
<code>fox2db_model.py</code>	math reference + limits + self-tests
<code>validate_model.py</code>	DB validation (one-step-ahead)